DOTE 2030 Midterm Cheatsheet · Lectures 2–5

How to read: blue box = formula to use · grey box = worked example · orange box = trap / common mistake · "min" = minutes, "hr" = hours, "/hr" = per hour

0 · Symbols used on this sheet

Symbol	Meaning	Unit
I	average inventory = average number of units inside (queue + being served)	units
R	average throughput rate = units leaving per unit of time	units/hr
T	average flow time = average time one unit spends inside	hr or min
λ (lambda)	arrival rate = average number of arrivals per unit of time	customers/hr
μ (mu)	service rate of ONE server = customers one server can finish per unit of time if always busy	customers/hr
c	number of identical servers sharing one queue	count
ρ (rho)	utilization of the server(s) = λ/μ (one server) or $\lambda/(c\cdot\mu)$ (c servers)	0 to 1
A, S	A = time between two arrivals (interarrival time); S = service time of one customer	hr or min
C_A, C_S	coefficient of variation (CV) of A and of S = standard deviation ÷ mean	no unit
I_q, T_q	number waiting in the QUEUE and time waiting in the QUEUE (not counting service)	units, hr
I_s, T_s	number being SERVED and time being SERVED ($T_s = 1/\mu$)	units, hr
p_i, w_i, C_i, W_i	job i: processing time, weight (priority), completion time, waiting time	min

1 · Process mapping: words you must know (L2)

- **Process**: the steps that turn input into output. **Flow unit**: the thing that moves through (customer, order, patient, rock, car).
- **Activity** (drawn as □): a step that takes time and needs at least one resource. **Buffer** (△): a place where flow units wait (a queue). **Decision point** (◇): flow goes to one of several paths (e.g. meat or mushroom pasta). **Arrow**: direction of flow.
- **Resource**: what does the work and has limited capacity: workers, machines, ovens, toasters, cranes, beds, operating rooms. One resource can do several activities. One activity can need several resources at once.
- **Resource pool**: identical resources that do the same activity, so a unit can go to any of them (two cashiers; Worker 1 or Worker 2 doing Activity A).
- **Flow chart** = boxes and arrows (what happens, in what order). **Gantt chart** = each resource's busy time on a time line; shows that the next unit can start before the previous unit finishes.
- There is no single correct map. Map the process the way it **actually** works and check it with the people who do it.

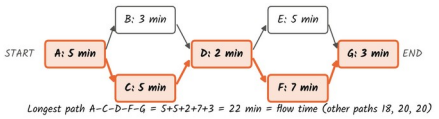
2 · Flow time (L2 p.19–21)

Flow time = the SHORTEST time one unit needs (one activity, or START → END)

$$\text{Flow time} = \sum \text{activity times} \quad \text{(one line)}$$

$$\text{Flow time} = \max_{\text{routes}} \left(\sum \text{times on route}\right) \quad \text{(parallel)}$$

longest route



Longest path A–C–D–F–G = 5+5+2+7+3 = 22 min = flow time (other paths 18, 20, 20)

- This is the **theoretical** (perfect-world) flow time: no waiting, no breakdowns. A real unit can take **longer** because it may wait in buffers.
- **Parallel paths, step by step**: (1) write down every route from START to END; (2) add the times on each route; (3) the biggest total is the flow time. (Parallel activities must use different resources, otherwise they cannot really run at the same time.)
- **Pooled activity** (Worker 1 OR Worker 2 does Activity A): each unit does A only **once**, so count A's time **once**.

Example (HW1 Q1): A(5) → {B(3) and C(5) at the same time} → D(2) → {E(5) and F(7) at the same time} → G(3).
Routes: A-B-D-E-G = 18; A-B-D-F-G = 20; A-C-D-E-G = 20; A-C-D-F-G = 22. Flow time = **22 min** (longest route).

Example (Review SA6, burger meal): place order 1 → {burger: grill 5 + cheese 1 + dressing 1 = 7} and {fries: cut 1 + fry 3 = 4} at the same time → assemble 1. Flow time = 1 + 7 + 1 = **9 min** (NOT 13: do not add both branches).

Example (in-class): queue → A (10 min, Worker 1 or Worker 2) → B 5 → C 2 → D 3 → E 5. Flow time = 10 + 5 + 2 + 3 + 5 = **25 min**.

Trap: "Flow time is 10 min, so every unit takes 10 min" is FALSE. Flow time is the minimum possible time.

3 · Unit load, capacity rate, bottleneck (L2)

$$\text{Unit load} = \sum \text{times of ALL activities the resource does}$$

(min per unit)

$$\text{Capacity} = \frac{60}{\text{unit load (min)}} \text{ per hr} = \frac{3600}{\text{unit load (s)}} \text{ per hr}$$

$$\text{Pool capacity} = n \times \text{capacity of one}$$

$$\text{Process capacity} = \min\{\text{capacities of all resources/pools}\}$$

= the bottleneck's capacity

Bottleneck = the resource (or pool) with the smallest capacity

How to find the bottleneck (5 steps)

- **Step 1**: list EVERY resource: every worker, machine, tool, oven, crane. (Activities are NOT resources.)
- **Step 2**: for each resource, add up the times of every activity it is needed for. If an activity needs two resources (e.g. "Crane and Employee 1"), its time goes into BOTH unit loads.
- **Step 3**: capacity = 60 ÷ unit load (per hour). **Step 4**: for a pool, multiply by the number of identical resources.
- **Step 5**: the smallest capacity is the process capacity; that resource is the bottleneck. Two resources can tie (both are bottlenecks).

McDonald's (seconds per burger): cashier 8, Worker 1 + toaster 10, Worker 2 8, Worker 3 6, Worker 4 2, Worker 5 2.
Capacity = 3600 ÷ time: 450, **360**, 450, 600, 1800, 1800 per hr ⇒ bottleneck = toaster (Worker 1); process capacity = **360 burgers/hr**.
Flow time = 8 + 10 + 8 + 6 + 2 + 2 = 36 s. **Wrong**: "3600 ÷ 36 = 100 burgers/hr". Several burgers are in progress at once, so capacity comes from the bottleneck, not from the flow time.
Add a 2nd cashier: cashier pool = 900/hr, but process capacity is **still 360** (cashier was not the bottleneck); flow time still 36 s.
Add a 2nd toaster: toaster pool = 720/hr ⇒ new capacity = min(450, 720, 450, …) = **450/hr**; new bottlenecks = cashier AND Worker 2 (tie).
Reassign: Worker 3 does patties + package + deliver = 6 + 2 + 2 = 10 s ⇒ 360/hr. Capacity unchanged, and Workers 4 and 5 are freed.

HW1 Q4 (car): cut metal 30 (saw), assemble body 40 (Employee 1), install engine 10 (crane AND Employee 1), quality check 60 (Employee 2).
Unit loads: saw 30, Employee 1 = 40 + 10 = **50**, crane 10, Employee 2 60 ⇒ capacities 2, 1.2, 6, **1** cars/hr ⇒ bottleneck **Employee 2**.
Hire Employee 3 for quality check: pool = 2 × 1 = 2/hr ⇒ new bottleneck **Employee 1** (1.2/hr).

Trap (Review T/F 11): you cut wood with a handsaw (2 hr) and nail (1 hr); partner sands (1 hr). Bottleneck = **you** (2 + 1 = 3 hr), NOT the handsaw. People are resources too.

4 · Making the process faster (L2–L3)

- Only the **bottleneck** limits capacity. Two ways to help it: (a) **pool**: add another copy of the bottleneck resource; (b) **cut its unit load**: make its tasks faster, or move some of its tasks to a resource with spare time.
- Speeding up a task done by a **non-bottleneck** resource: flow time goes down, capacity does **not** change.
- Fixing one bottleneck can create a **new** bottleneck. Cut a bottleneck's unit load only until it equals the **next** smallest capacity; after that, more cutting gives no extra capacity.

Review SA2: Worker does A 10 + C 15 = 25 min (2.4/hr); Machine B 12 min (5/hr); Machine C 15 min (4/hr); 2 sales reps, 20 min each (pool 2 × 3 = 6/hr). Bottleneck = Worker (2.4/hr). Useful cut = until Worker equals Machine C's 15 min ⇒ **10 min**. (Not 5 min: the reps are a pool of 6/hr, not 3/hr.)
Pizza (Overcooked, L3): 2 chefs, 1 oven, 22 s of chef work per pizza. Oven unit load = load 1 + bake 10 + unload 1 = 12 s ⇒ oven capacity 60/12 = **5 pizzas/min**. No way of dividing work between the chefs can beat 5.
Chef A 14 s, Chef B 8 s ⇒ 60/14 = 4.29/min (Chef A is the bottleneck). Balanced: A 12 s, B 10 s ⇒ 5/min (+16.7%), same 22 s of total work. Full pooling: 2 × 60/22 = 5.45 ⇒ still capped at 5 by the oven.

5 · Throughput, utilization, implied utilization (L2)

$$\text{Throughput} = \min\{\text{input rate, process capacity}\}$$

$$\text{Utilization} = \frac{\text{throughput}}{\text{resource capacity}} \le 100\% \quad \text{uses THROUGHPUT}$$

$$\text{Implied utilization} = \frac{\text{input rate}}{\text{resource capacity}} \quad \text{(can be } > 100\%\text{)} \quad \text{uses INPUT}$$

$$\text{Cycle time} = \frac{1}{\text{throughput}} \quad \text{time between two finished units}$$

If you see…	It means…
every utilization < 100%	demand is the limit: input < process capacity
one utilization = 100%	that resource is the bottleneck; input ≥ capacity
implied utilization > 100%	input > that resource's capacity ⇒ inventory (queue) builds up
implied utilization = 100%	input exactly equals capacity ⇒ no build-up

Review p.3–4: A (10 min) done by Worker 1 OR Worker 2 (pool 2 × 6 = 12/hr); B S + C 2 by Worker 3 (7 min ⇒ 60/7 = 8.57/hr); D 3 min by Worker 4 (20/hr). Input 10/hr.
Throughput = min(10, 8.57) = **8.57/hr**. Utilization = 8.57 ÷ capacity: W3 100%, W4 43%, pool 71%.
Implied utilization = 10 ÷ capacity: W3 117%, W4 50%, pool 83.33%.
HW1 Q5: R1 does A 6 + B 2 = 8 min (7.5/hr); R2 does B 2 + C 4 = 6 min (10/hr); R3 does D 4 min (15/hr). Input 2/hr ⇒ throughput 2 ⇒ utilization of R2 = 2/10 = **20%**.
Review SA1(b): capacity 3/hr ⇒ cycle time = 1/3 hr = **20 min per unit**.

Traps: input > capacity ⇒ *implied* utilization > 100%, utilization is still ≤ 100%. Throughput = capacity does NOT mean inventory builds (input may equal capacity exactly).

6 · Profit questions: rent / hire / invest (L2, Review)

- **Steps**: (1) find the bottleneck; adding a non-bottleneck resource adds NO output. (2) Recompute all capacities with the change; the new minimum is the new capacity. (3) Throughput = min(input, new capacity). (4) Profit per hour = throughput × money per unit − all hourly costs. (5) Compare with the current profit.

Review SA1(e): A 10, B 20, C 15 min ⇒ 6, **3**, 4 per hr. Each unit earns $10; an extra resource costs $14/hr.
Now: 3 × $10 = **$30/hr**. Rent a 2nd B: B pool = 6/hr, C becomes the bottleneck (4/hr) ⇒ 4 × $10 − $14 = **$26/hr** < $30 ⇒ **do not rent**.
Review SA11(c) bank: Worker 1 does 4 + 2 = 6 min (10/hr); Worker 2 does 3 + 7 = 10 min (6/hr); computer 2 min (30/hr). Input 10/hr, $10 per customer, $20/hr per worker.
Now: 6 × $10 − $40 = **$20/hr**. Hire Worker 3 to take activity 3 or 4 ⇒ the slower of the two workers has 7 min ⇒ 8.57/hr ⇒ $85.7 − 60 = **$25.7/hr** ⇒ **hire**.
HW1 Q4(c): $10 per minute cut; want capacity 2 cars/hr ⇒ every unit load ≤ 60/2 = 30 min.
Employee 2: quality check 60 → 30 = 30 min × $10 = **$300**. Employee 1: assemble + 10 (engine, cannot change) ≤ 30 ⇒ assemble 40 → 20 = 20 min × $10 = **$200**. Total **$500**.

7 · More process examples (Review)

Problem	Unit loads ⇒ capacity per hr	Answer
In-class / SA7	Workers 1+2 pool on A: 2 × 6 = 12; Worker 3 does B+C+D = 5+2+3 = 10 min ⇒ 6; Worker 4 does E 5 min ⇒ 12	Bottleneck Worker 3, 6/hr. At throughput 3/hr: utilization 25%, 50%, 25%
SA1(f)	Activity A now also needs resource C ⇒ C = 10 + 15 = 25 min ⇒ 2.4	Bottleneck C (2.4/hr). Speed C up by 5 ⇒ 20 min ⇒ 3/hr
SA5	Prof: 10 + 8 + 7 = 25 min ⇒ 2.4; TA 12.5 min ⇒ 4.8	Bottleneck Prof, 2.4/hr
SA6	Cashier 1+1 = 2 (30); Worker 1 1+3 = 4 (15); fryer 3 (20); Worker 2 5+1+1 = 7 (8.57); grill 5 (12)	Bottleneck Worker 2, 8.57/hr. Input 10 ⇒ throughput 8.57. Input 5 ⇒ Worker 2 utilization 58.33%
SA11(a)(b)	Worker 1 6 min (10); Worker 2 10 min (6); computer 2 min (30)	Capacity 6/hr. Arrivals every 5 min = 12/hr ⇒ implied utilization of Worker 1 = 120%

8 · Inventory build-up over time (L3)

- Inventory **in the process** = units waiting in buffers + units being worked on (pizza: 3 waiting + 1 in prep + 1 in oven = 5). Inventory can never be below 0.

$$Q_{end} = \max\{0,\ Q_{start} + \text{input} - \text{capacity}\}$$

discrete table, each period

$$\text{Slope} = \text{input rate} - \text{capacity rate} \quad \text{(while queue} > 0\text{)}$$

continuous graph

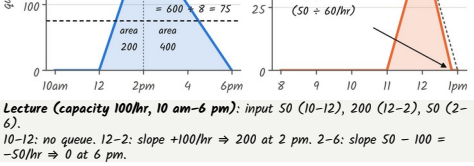
$$\text{Time to empty} = \frac{\text{queue size}}{\text{capacity} - \text{input}}$$

$$\text{Discrete average} = \frac{\sum \text{end of period queues}}{\text{number of periods}}$$

$$\text{Continuous average} = \frac{\text{area under graph}}{\text{total time}}$$

include zero stretches

- While there is a queue, output = capacity. With no queue, output = input. Average throughput = total served ÷ total time.
- **Stable process**: long-run average input ≤ capacity (otherwise the queue grows forever). Short periods with input > capacity still create temporary queues.
- **Continuous, step by step**: (1) inventory at each time the rate changes; (2) if the queue shrinks, find the exact time it hits 0; (3) cut the area into triangles (½ × base × height) and rectangles (base × height); (4) divide by the WHOLE time window, including the zero parts.



Lecture (capacity 100/hr, 10 am–6 pm): input 50 (10–12), 200 (12–2), 50 (2–6).
10–12: no queue. 12–2: slope +100/hr ⇒ 200 at 2 pm. 2–6: slope 50 − 100 = −50/hr ⇒ 0 at 6 pm.
Area = ½·2·200 + ½·4·200 = 200 + 400 = 600 ⇒ average = 600 ÷ 8 hr = **75 customers**. Served = 2×50 + 6×100 = 700 ⇒ R = 87.5/hr.
HW1 Q3: +20/hr for 2 hr (0 → 40), flat 2 hr, −20/hr for 1 hr (40 → 20).
Area = triangle ½·2·40 = 40 + rectangle 2·40 = 80 + (rectangle 1·20 = 20 + triangle ½·1·20 = 10) = 150 ⇒ 150 ÷ 5 = **30 people**.
SA3: start 65, +4/hr for 5.5 hr ⇒ end 65 + 22 = 87. Area = 65·5.5 + ½·5.5·22 = 357.5 + 60.5 = 418 ⇒ 418 ÷ 5.5 = **76**.
SA9: start 50, +5/hr for 4 hr ⇒ area 50·4 + ½·4·20 = 240 ⇒ **60**.
SA4 (discrete): input 15, 32, 39, 10 per hr, capacity 30. End queues: max(0, 15−30) = 0; 0+32−30 = 2; 2+39−30 = 11; max(0, 11+10−30) = 0 ⇒ (0+2+11+0) ÷ 4 = **3.25**.
SA12(a) weekly tank (graph given): tons in tank rise 0 (Sun) → 25 (Wed), then 25 → 75 (Fri), stay 75 until Sat, fall to 0 (next Sun). Areas: triangle ½·3·25 = 37.5; rectangle under 25 Wed–Sat 3·25 = 75; triangle ½·2·50 = 50; rectangle Fri–Sat 1·50 = 50; triangle Sat–Sun ½·1·75 = 37.5. Total 250 ⇒ 250 ÷ 7 days = **35.71 tons**.
Airport (discrete, L3): security can screen 15 passengers per 15-min slot, 6–10 am (16 slots). Arrivals per slot: 7, 10, 8, 12, 9, 16, 14, 19, 22, 17, 13, 11, 12, 8, 10, 7 (total 195 < capacity 240). Queue rule max(0, prev + arrivals − 15) gives 0,0,0,0,0, 1, 0, 4, 11, 13, 11, 7, 4, 0,0,0 ⇒ average 51 ÷ 16 = **3.19**. Enough TOTAL capacity still gives queues when some slots exceed 15.
Same data, two methods (L3 summary): inventory starts at 0, rises steadily to 20 by end of week 1, then to 50 by end of week 2. Discrete (only end-of-week values 20 and 50): (20 + 50) ÷ 2 = **35**. Continuous (area under the rising line): week 1 triangle ½·1·20 = 10; week 2 rectangle 1·20 = 20 + triangle ½·1·30 = 15 ⇒ 45 ÷ 2 = **22.5**. Use the method the question names.

Review SA8 (hospital), answer key says 10, careful method gives 9.17: capacity 150/hr. 11–12: input 200 ⇒ +50/hr ⇒ 50 patients at 12. 12–1: input 90 ⇒ slope 90 − 150 = −60/hr ⇒ empty after 50/60 hr = **12:50**, not 1:00.
Area = ½·1·50 + ½·(50/60)·50 = 25 + 20.83 = 45.83 ⇒ 45.83 ÷ 5 hr = **9.17 patients**. Write the 12:50 step so the grader sees your reasoning.

9 · Little's Law (L3)

$$I = R \times T \qquad R = \frac{I}{T} \qquad T = \frac{I}{R}$$

I = avg inventory, R = flow time

- Works only for a **stable** process, with long-run averages.
- I, R and T must describe the **same part** of the process: whole process ($I = R{\cdot}T$), queue only ($I_q = R{\cdot}T_q$), or service only ($I_s = R{\cdot}T_s$). And use the **same time unit** for R and T.

BBA: 850 graduate/yr, 4 yr each ⇒ I = 850 × 4 = 3400 students. 3500 students, 4 yr ⇒ R = 875/yr. 3600 students, 800/yr ⇒ T = 4.5 yr.
HW1 Q2(a): R = 12 rocks/hr, T = 10 min = 1/6 hr ⇒ I = 12 × 1/6 = **2 rocks**.
HW1 Q2(b): I = 4 waiting + 1 being crushed = 5 rocks (T covers the whole process, so I must too); T = 20 min ⇒ R = 5 ÷ 20 = **0.25 rocks/min**.
Review SA7(d): R = 3/hr, T = 30 min = 0.5 hr ⇒ I = **1.5 units**. **T/F 13**: I = 100, T = 5 hr ⇒ R = 100/5 = **20/hr** (not 500).
Queue waiting time (same 10 am–6 pm lecture example): average queue I_q = 75 people, throughput R = 700 served ÷ 8 hr = 87.5/hr ⇒ T_q = 75 ÷ 87.5 = 0.857 hr = **51.4 min**.

- **Heathrow**: count people entering and leaving the security area ⇒ number inside over time (I) and exit rate (R) ⇒ average time $T = I \div R$, without tracking each person.
- **Uber**: L drivers, O of them idle ⇒ busy drivers $L - O = Y \times (\eta + T)$, where Y = trips per minute, η = pickup time, T = ride time ⇒ $Y = (L - O) \div (\eta + T)$. Shorter pickup times ⇒ more trips.

10 · Shouldice Hospital case (L3)

- Each patient stays exactly 3 nights; surgery on day 2. Admits 30/day Sunday–Thursday. No surgery Saturday/Sunday. 12 surgeons × 4 operations/day. 5 operating rooms (ORs) × 8 hr/day, 1 hr per operation. 90 beds.

Resource	Capacity per week	Limit per day
Beds	90 beds × 7 nights ÷ 3 nights per patient = 210	≤ 90 patients per night
Surgeons	12 × 4 × 5 days = 240	≤ 48 operations
ORs (bottleneck)	5 × 8 hr × 5 days ÷ 1 hr = **200**	≤ **40** operations

- Now: R = 5 × 30 = **150/week**. Patients per night Sun–Sat: 30, 60, 90, 90, 90, 60, 30 ⇒ average I = (30·2 + 60·2 + 90·3) ÷ 7 = 450/7 ≈ **64**. Check: I = R × T = 150 × 3/7 ≈ 64 ✓.

Plan to reach 180/week	Peak beds	Works?
Surgery also on Saturday (admit 30/day Sun–Fri)	90	Yes, but surgeons refuse Saturdays
Admit 36 each day Sun–Thu	108	Needs 18 more beds
Admit 40, 40, 20, 40, 40	100	Needs 10 more beds
Admit 45, 45, 0, 45, 45	90	45 operations > 40 ⇒ needs 1 more OR
No new resources: 40, 30, 20, 40, 30	90	160/week, feasible
No new resources: **40, 40, 10, 40, 40**	90	**170/week = the maximum**

- Why 170 is the maximum: $x_1,\dots x_5$ = operations Mon–Fri. ORs: each $x \le 40$. Beds on Wednesday night: $x_2 + x_3 + x_4 \le 90$. So total = $x_1 + (x_2 + x_3 + x_4) + x_5 \le 40 + 90 + 40 = 170$.

11 · Queueing systems: structures & policies (L4)

- Queue design = **structure** (how queues and servers connect) + **service policy** (who is served next). Together they decide waiting time, access and match quality.
- **Structures**: one queue one server (ATM) · parallel servers with ONE shared queue (bank) or SEPARATE queues (supermarket) · servers in series/branches (airport check-in → security → gate) · flexible routing (Disneyland) · two-sided matching (Uber riders + drivers) · one-sided matching (online game players) · resource allocation (kidneys to patients).
- **Policies**: who is served (arrival order or priority) · which server (take turns, or least work) · who is matched · when to act (now, or wait and batch). With one server: first come first served (FCFS), shortest processing time (SPT), priority.

12 · Scheduling jobs: SPT and WSPT (L4)

$$C_i = W_i + p_i \qquad \sum C_i = \sum W_i + \sum p_i$$

$\sum p$ is the same for every order

$$SPT:\ \text{order by smallest } p_i \qquad WSPT:\ \text{order by smallest } \frac{p_i}{w_i}$$

SPT minimises total completion and total waiting; WSPT minimises the weighted totals $\Sigma w_i C_i$ and $\Sigma w_i W_i$

- Assumes: all jobs ready at time 0, one server, processing times known, no interruptions.
- **Why shortest first**: the 1st job's time is inside every later completion time (counted n times); the last job's time is counted once.
- **Steps**: (1) sort; (2) C_i = running total of the ACTUAL processing times in that order; (3) total weighted = $\Sigma C - \Sigma p$; weighted = $\Sigma w_i C_i - \Sigma w_i p_i$.

SPT: p = 6, 5, 8, 15 (patients 1–4) ⇒ order 2, 1, 3, 4 ⇒ C = 5, 11, 19, 34 ⇒ ΣC = **69 min**; Σw = 69 − 34 = **35 min**. (Check: 4×5 + 3×6 + 2×8 + 1×15 = 69.)
Two orders for p = 10, 15, 7: order 1-2-3 ⇒ C 10, 25, 32 ⇒ ΣC 67, ΣW 35. Order 3-1-2 (SPT) ⇒ C 7, 17, 32 ⇒ ΣC **56**, ΣW **24**.
WSPT: p = 6, 5, 8, 15 with weights 4 (high), 1 (low), 2 (medium), 1 (low) ⇒ p/w = 1.5, 5, 4, 15 ⇒ order **1, 3, 2, 4**.

Patient	p	w	p ÷ w	Rank	C (running total)	w × C
2	5	4	1.25	1	5	20
7	10	4	2.5	2	5+10 = 15	60
9	13	4	3.25	3	15+13 = 28	112
5	15	4	3.75	4	28+15 = 43	172
1	6	1	6	5	43+6 = 49	49
4	8	1	8	6	49+8 = 57	57
8	9	1	9	7	57+9 = 66	66
6	19	2	9.5	8	66+19 = 85	170
3	20	2	10	9	85+20 = 105	210
10	17	1	17	10	105+17 = 122	122
Total						**1038**

$\Sigma wp_i = 4{\cdot}5 + 4{\cdot}10 + 4{\cdot}13 + 4{\cdot}15 + 6 + 8 + 9 + 2{\cdot}19 + 2{\cdot}20 + 17 = 290 \Rightarrow \Sigma w_i w_i = 1038 - 290 = \mathbf{748}$.

Trap: rank by $p \div w$, but compute C by adding the real p (not the ratios).

13 · Why variability matters (L4)

All three ATMs: on average 0.5 arrivals per minute and 1 minute of service. Different patterns give different queues:

ATM pattern	Waits	Avg wait	Avg queue	Avg flow time
Regular: arrive every 2 min, 1 min service	0, 0, 0, 0	0	0	1 min
Clustered: pairs arrive at 4, 4, 8, 8	0, 1, 0, 1	0.5 min	0.25	1.5 min
Variable service 2.5, 0.5, 0.5, 0.5	0, 0.5, 0, 0	0.125 min	0.0625	1.125 min

Average queue = arrival rate × average wait (Little's Law). **Averages alone do not determine the queue; variability does.**

14 · Probability tools (L4)

- **Random variable**: a number for an uncertain outcome. **Discrete**: can be counted (number of arrivals in 10 min: 0, 1, 2, …). **Continuous**: any value in a range (time between arrivals, service time).

$$E[X] = \sum_i x_i \cdot P(X = x_i)$$ *the prediction*

$$Var(X) = E\left[(X - E[X])^2\right] = \sum_i (x_i - E[X])^2 \cdot P(X = x_i)$$ *avg squared error (units²)*

$$\sigma = \sqrt{Var(X)} \qquad CV = \frac{\sigma}{E[X]}$$ *σ in original units; CV has no unit*

Items per order: 1 (prob 0.25), 3 (0.5), 5 (0.25). $E = 1{\cdot}0.25 + 3{\cdot}0.5 + 5{\cdot}0.25 =$ **3**.
Errors −2, 0, +2 average to 0 (they cancel), so square them: $Var = 0.25{\cdot}4 + 0.5{\cdot}0 + 0.25{\cdot}4 =$ **2**; *$\sigma = \sqrt{2} =$* **1.41**; *$CV = 1.41 \div 3 =$* **47.1%**.
Same σ = 100: mean 1,000,000 ⇒ CV = 0.01% (tiny); mean 1,000 ⇒ CV = 10% (big).

	Poisson	Exponential	Constant (deterministic)
Describes	a COUNT: number of arrivals in a time window	a TIME: gap between arrivals (rate λ) or service time (rate μ)	a fixed time
Mean	m (e.g. $\lambda \times$ window)	$1/\lambda$ (or $1/\mu$)	the value
Variance	m (variance = mean)	$1/\lambda^2$	0
Std dev	$\sqrt{m}$	$1/\lambda$ (std dev = mean)	0
CV	$1/\sqrt{m}$	**1**	**0**

λ = 12 arrivals/hr ⇒ arrivals in 10 min ~ Poisson(12 × 10/60 = 2): mean 2, variance 2, std dev 1.41, CV 0.707.
λ = 12/hr, μ = 30/hr, both exponential: time between arrivals mean = std dev = 60/12 = 5 min (variance 25 min²); service mean = std dev = 60/30 = 2 min (variance 4 min²); both CV = 1.
T/F 17: *Poisson arrivals at 15/hr ⇒ gaps are exponential ⇒ std dev of gap = mean = 1/15 hr =* **4 min** *(TRUE).*

- Poisson arrivals and exponential gaps between arrivals go together (each implies the other). Exponential is popular because it often fits reality, its CV = 1 is easy, and it gives Poisson counts.

Trap (T/F 16): *mean = std dev = 10 min does NOT prove the times are exponential. You must be told (or assume) it.*

15 · Single-server queue (L4)

- Picture: *arrivals (rate λ)* → **buffer / queue** → **one server** (rate μ) → leave. Stable only if $\lambda < \mu$ (i.e. $\rho < 1$); then throughput = λ.

$$\rho = \frac{\lambda}{\mu} < 1$$ *utilization; must be < 1 (stable)*

$$l_q \approx \frac{\rho^2}{1-\rho} \times \frac{C_A^2 + C_S^2}{2}$$ *people WAITING (not being served)*

$$T_q = \frac{l_q}{\lambda} \qquad T_s = \frac{1}{\mu} \qquad l_s = \frac{\lambda}{\mu} = \rho$$

$$l = l_q + \rho \qquad T = T_q + \frac{1}{\mu} = \frac{l}{\lambda}$$ *whole system = waiting + being served*

$$M/M/1:\ l_q = \frac{\rho^2}{1-\rho} = \frac{\lambda^2}{\mu(\mu-\lambda)} \qquad M/D/1:\ l_q = \frac{\rho^2}{2(1-\rho)}$$

- First bracket = **utilization effect** (from the averages). Second bracket = **variability effect** (from C_A, C_S). l_q does NOT include the person being served.
- C_A = std dev of time between arrivals ÷ mean time between arrivals. C_S = std dev of service time ÷ mean service time. Use TIMES (not rates) and the same units.

Model	C_A	C_S	$(C_A^2+C_S^2)\div 2$	Number waiting l_q
G/G/1	σ÷mean	σ÷mean	compute it	full PK formula
M/M/1	1	1	1	$\rho^2/(1-\rho) = \lambda^2/[\mu(\mu-\lambda)]$ (exact)
M/D/1	1	0	1/2	$\rho^2/(1-\rho) \times 1/2 = \lambda^2/[2\mu(\mu-\lambda)]$
D/D/1	0	0	0	0 (no queue)
G/M/1	σ÷mean	1	$(C_A^2+1)/2$	PK formula
D/G/1	0	σ÷mean	$C_S^2/2$	PK formula

- **Name code "X/Y/c"**: X = how arrivals behave, Y = how service times behave, c = number of servers. M = exponential (CV 1), D = deterministic / constant / "exactly" (CV 0), G = general (use the given CV).

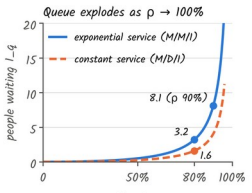
Solving any queue question (6 steps)

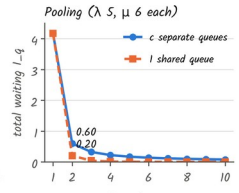
- (1) Convert everything to ONE time unit (usually per hour). (2) λ and μ ⇒ ρ; check ρ < 1. (3) C_A and C_S: "exponential" ⇒ 1, "exactly / constant" ⇒ 0, otherwise std dev ÷ mean. (4) l_q from PK. (5) $T_q = l_q \div \lambda$, then convert hours → minutes (× 60). (6) If asked "in the system" or "including service": add ρ (people) or $1/\mu$ (time).

Restaurant 1: *λ = 4/hr; mean service 10 min ⇒ μ = 6/hr; both exponential ($C_A = C_S = 1$). ρ = 4/6 = 2/3.*
$l_q = (4/9) \div (1/3) \times 1 =$ **4/3 ≈ 1.33 customers**; *$T_q = (4/3) \div 4 = 1/3$ hr =* **20 min**.
Restaurant 2: *λ = 4/hr, std dev of gap 5 min; service mean 10 min, std dev 3 min. Mean gap = 60/4 = 15 min ⇒ $C_A = 5/15 = 1/3$; $C_S = 3/10 = 0.3$.*
$l_q = (4/3) \times (0.111 + 0.09) \div 2 =$ **0.134**; *$T_q = 0.134 \div 4 = 0.034$ hr ≈* **2 min**.
Less variability ⇒ 10× shorter queue.
M/D/1: *λ = 4/hr, service exactly 10 min (μ = 6, $C_S = 0$): $l_q = 4^2 \div (2{\cdot}2) =$* **2/3**; *$T_q = 1/6$ hr =* **10 min** *(half of M/M/1).*
Office hours: *λ = 10/hr. M/M/1 with μ = 20: ρ = 1/2, $l_s = (1/4) \div (1/2) =$* **1/2**, *$T_q = 1/20$ hr =* **3 min**. *Exactly 3 min each (M/D/1): $l_q =$* **1/4**, *$T_q = 1/40$ hr =* **1.5 min**.
Review drive-through (M/M/1): *λ = 20/hr, 2 min per car ⇒ μ = 30/hr ⇒ ρ = 2/3.*
Cars waiting $l_q = 4/3$. Cars in system $l = 4/3 + 2/3 =$ **2**. *Wait $T_q = (4/3) \div 20$ hr =* **4 min**. *Time at bank $T = 2 \div 20$ hr =* **6 min**.
Constant service (M/D/1): $l_q = 2/3 \Rightarrow l = 2/3 + 2/3 = 4/3 \Rightarrow T = (4/3) \div 20$ hr = **4 min**.
Review airport, 3 separate lines *(each one agent, 2 min ⇒ μ = 30/hr): Diamond λ10: ρ 1/3, l_q 1/6, $l = 1/2 \Rightarrow T =$* **3 min**. *Silver λ20: ρ 2/3, $l = 2 \Rightarrow$* **6 min**. *Regular λ28: ρ 14/15, $l_q = 196/15 = 13.07$, $T_q = 28$ min, $T =$* **30 min**.
Average over all 58 flyers/hr (weight by arrival share): $(28{\cdot}30 + 20{\cdot}6 + 10{\cdot}3) \div 58 =$ **17.07 min**.
Review homework queue (D/G/1): *10 questions/day exactly ($C_A = 0$). Each takes 1 hr = 1/24 day (μ = 24/day), std dev ½ hr ⇒ $C_S = 0.5$. ρ = 10/24 = 5/12.*
$l_q = (25/144) \div (7/12) \times (0 + 0.25) \div 2 =$ **25/672**; *$T_q = (25/672) \div 10 = 25/6720 \approx$* **0.0037 days**.

16 · Utilization vs variability, and the OM triangle

Utilization ρ (M/M/1)	50%	80%	90%	95%
Number waiting $l_q = \rho^2/(1-\rho)$	0.50	3.20	8.10	18.05





- The queue **explodes** as ρ → 100% (80% → 90% more than doubles it). With no variability, high utilization is good; with variability, very high utilization is BAD.
- More variability (bigger C_A or C_S) ⇒ longer queue at every utilization. At ρ = 80%: exponential service 3.2 waiting vs constant service 1.6.

Demand 8/hr (exponential arrivals)	Service	μ	ρ	l_q	Wait T_q
Current	6 min, exponential	10	80%	3.20	24 min
More capacity	5 min, exponential	12	66.7%	1.33	10 min
Less variability	6 min, constant	10	80%	1.60	12 min

- *To reach $l_q = 1.6$: exponential service needs $\rho^2/(1-\rho) = 1.6 \Rightarrow \rho = 0.697 \Rightarrow \mu = 8 \div 0.697 =$* **11.48/hr**; *constant service needs only* **10/hr**. *Less variability saves capacity.*
- ***OM triangle*** *(3 ways to cope with variability):* **Capacity**: *add servers (costs money, idle time)* · **Inventory**: *let a queue/buffer form (costs customers' waiting time)* · **Information**: *reduce variability, e.g. schedule appointments (costs effort). Clinic: add a doctor / let patients wait / plan slots by visit reason.*

17 · Multi-server queue: c servers, ONE shared queue (L5)

- *One queue, first come first served, c identical servers each with rate μ. Stable only if $\lambda < c\mu$. Throughput = λ.*

$$\rho = \frac{\lambda}{c\mu} < 1$$ *same for the pool and each server*

$$l_q \approx \frac{\rho^{\sqrt{2(c+1)}}}{1-\rho} \times \frac{C_A^2 + C_S^2}{2}$$ *c = 1 gives the single-server formula*

$$T_q = \frac{l_q}{\lambda} \qquad l_s = c\rho = \frac{\lambda}{\mu} \qquad T = T_q + \frac{1}{\mu}$$ *l_s = number of busy servers*

Servers c	1	2	3	4	5	6
Power $\sqrt{2(c+1)}$	√4 = 2	√6 = 2.449	√8 = 2.828	√10 = 3.162	√12 = 3.464	√14 = 3.742

- *With c = 1 the power is 2, so this is the single-server formula. M/M/c (both exponential): $l_q \approx \rho^{\sqrt{2(c+1)}} \div (1-\rho)$.*
- ***Pooling rule***: *ONE shared queue for c servers always has a SHORTER total queue than c SEPARATE single-server queues with the same total demand and capacity. Separate: $c \times \rho^2/(1-\rho) \times (\ldots)$. Shared: $\rho^{\sqrt{2(c+1)}}/(1-\rho) \times (\ldots)$.*

CUHK card office: *λ = 90/hr; 3 min per student ⇒ μ = 20/hr; c = 5; exponential. ρ = 90 ÷ (5 × 20) = 0.9. Power $\sqrt{12} = 3.464$: $0.9^{3.464} = 0.694$. $l_q = 0.694 \div 0.1 \times 1 =$* **6.94 students**; *$T_q = 6.94 \div 90 = 0.077$ hr =* **4.63 min**.
Bank, 2 separate queues (M/M/1 each): *queue 1 λ5, μ10 ⇒ 2S ÷ (10 × 5) = 1/2; queue 2 λ15, μ20 ⇒ 22S ÷ (20 × 5) = 9/4 ⇒ total* **2.75 waiting**.
Bank, 1 shared queue (M/M/2): *λ = 20, μ = 15, c = 2 ⇒ ρ = 20/30 = 2/3; $l_q = (2/3)^{2.449} \div (1/3) =$* **1.11 waiting** *(much better).*
Review airport merged (M/M/3): *λ = 58, μ = 30, c = 3 ⇒ ρ = 58/90 = 0.644; $0.644^{2.828} = 0.289$; ÷ (1 − 0.644 = 0.356) ⇒ $l_q =$* **0.812**.
$T_q = 0.812 \div 58 = 0.014$ hr = 0.84 min; $T = 0.84 + 2 =$ **2.84 min** *(separate lines averaged 17.07 min).*
Write the PK formula for a name (Review SA12e): *D/M/2: $\rho^{\sqrt{6}}/(1-\rho) \times (0+1)/2$ · G/M/1: $\rho^2/(1-\rho) \times (C_A^2+1)/2$ · M/M/3: $\rho^{\sqrt{8}}/(1-\rho)$ · M/G/c: $\rho^{\sqrt{2(c+1)}}/(1-\rho) \times (1+C_S^2)/2$.*
Two independent queues (SA12d, restaurant): *drive-through: 8 orders/hr, 6 min each (μ = 10/hr), M/M/1 ⇒ $l_q = 8^2 \div (10 \times 2) = 16/5$, $T_q = (16/5) \div 8 = 2/5$ hr. Dining room: 20/hr, 2 min each (μ = 30/hr) ⇒ $l_q = \frac{4}{9} \cdot 20^2 \div (30 \times 10) = 4/3$, $T_q = 1/15$ hr.*
Total waiting = 16/5 + 4/3 = **68/15 ≈ 4.53**. *Average wait = (8/28)(2/5) + (20/28)(1/15) =* **17/105 hr ≈ 0.162 hr** *(weight by arrival share).*
Two queues in a row (SA12g, helpline): *calls arrive every 4 min on average (λ = 15/hr), std dev 1 min ⇒ $C_A = 1/4$. Queue 1: Operator 1 takes 60 calls/hr, exponential ($C_S = 1$) ⇒ ρ = 15/60 = 1/4 ⇒ $l_q = [(1/16) \div (3/4)] \times (1/16 + 1)/2 = 17/384$, $T_q = (17/384) \div 15 \approx 0.003$ hr.*
Queue 2 G/M/4: μ4 each ⇒ ρ = 15/16; $l_q = 0.937^{3.162} \div (1/16) \times (1/16 + 1)/2 \approx 6.93$, $T_q \approx 0.462$ hr. Total on hold = 0.003 + 0.462 ≈ **0.465 hr**.

18 · Matching systems: online games (L5, concepts)

- *Game matchmaking =* **one-sided** *matching: players wait in one pool and leave together once matched. (Uber = two-sided; a service queue ends when a server starts.)*
- *The platform decides* **when** *to match (now, or wait for more players) and* **whom** *to match. Goal: player engagement.*
- ***Suspense*** *= the next event could still change who you think will win (uncertain until the end).* ***Surprise*** *= an event suddenly changes your belief. Studies: Ely–Frankel–Kamenica 2015; CS:GO on Twitch (more suspense ⇒ viewers stay). ⇒ Match players of* **similar skill**.
- *Players rated 1000, 1400, 1010, 1390 arrive in order: match immediately ⇒ skill gaps 400 and 380 (short wait); wait for all four ⇒ gaps 10 and 10 (long wait, players may quit). Trade-off: match quality vs waiting time.*
- *Downsides of always balanced matches: long waits for top players; your opponents decide your win chance; win rate stays near 50% even as you improve.*

19 · True/False traps from the review (with the reason)

F	"Flow time 10 min ⇒ every unit takes 10 min"	flow time is the minimum; real units can wait
T	One bottleneck doing A, B, C; shorten A ⇒ process capacity rises	bottleneck's unit load falls
T	Hire a 2nd identical cashier (10/hr each) ⇒ cashier pool 20/hr	pool = number × one
T	Implied utilization > 100% ⇒ inventory builds	input > that capacity ≥ process capacity
F	Shorten ANY activity ⇒ process capacity rises	only if a bottleneck does it
T	Shorten an activity of resource R ⇒ R's capacity rises	R's unit load falls
F	Throughput = capacity ⇒ inventory builds, need a buffer	input may equal capacity exactly
F	Input > capacity ⇒ bottleneck utilization > 100%	implied utilization > 100%; utilization ≤ 100%
F	Capacity 12/hr, 10 arrive 6–7 pm ⇒ 7 pm queue shorter than 6 pm	6 pm queue might already be 0
F	Handsaw is the bottleneck	you (2 + 1 = 3 hr) are
F	Each activity has a capacity; min of these = process capacity	only resources have capacity
F	l = 100, T = 5 hr ⇒ R = 500/hr	$R = l \div T = 20$/hr
F	Server utilization goes up ⇒ queue l_q goes down	l_q goes up as ρ rises
T	D/D/1 ⇒ $l_q = 0$	$C_A = C_S = 0$
F	Mean = std dev = 10 min ⇒ exponential with λ = 1/10	other distributions share mean and std dev
T	Poisson 15 arrivals/hr ⇒ std dev of a gap = 4 min	exponential: std dev = mean = 1/15 hr
F	Gap variance up ⇒ more people being served	PK is about the queue; $l_s = \rho$ unchanged
F	Double λ and μ (same std devs) ⇒ l_q doubles	ρ same, both CVs double ⇒ l_q × 4

20 · Which tool answers which question?

The question asks for…	Use (section)
fastest time through the process	flow time; longest path if parallel (§2)
maximum output / what limits output	unit loads → capacities → smallest = bottleneck (§3)
actual output	throughput = min(input, capacity) (§5)
how busy a resource is	utilization (uses throughput) or implied utilization (uses input) (§5)
is it worth renting / hiring / investing	new bottleneck capacity → profit per hour (§6)
queue over a day with given rates	inventory build-up, discrete or continuous (§8)
average number / time / rate when two are known	Little's Law $l = R \times T$ (§9)
best order to serve jobs	SPT; with priorities WSPT using $p \div w$ (§12)
queue length or wait when times are random	PK formula, then $T_q = l_q \div \lambda$ (§15, §17)
separate lines vs one shared line	c × single-server PK vs multi-server PK (§17)
how to reduce waiting	OM triangle: capacity / inventory / information (§16)

21 · Unit conversions

- *Capacity per hour = 60 ÷ (minutes per unit) = 3600 ÷ (seconds per unit). Per minute → per hour: × 60.*
- *Mean time from a rate: 1 ÷ λ hours × 60 = minutes. 20/hr ⇒ 3 min; 4/hr ⇒ 15 min; 30/hr ⇒ 2 min; 12/hr ⇒ 5 min.*
- *Std dev in minutes but rates per hour: convert std dev to hours (5 min = 1/12 hr) before CV = std dev ÷ mean, or put both in minutes.*
- *Answer in hours × 60 = minutes (1/3 hr = 20 min; 0.077 hr = 4.63 min). 1 hr = 1/24 day.*

22 · Errors in the review's answer key

SA8 hospital: correct average **9.17** *(queue empties at 12:50); key says 10. SA1(a): the bottleneck is resource* **B** *(text says A). SA1(b): cycle time = 1/3* **hour per unit** *= 20 min. SA1(e): second case should read "if we* **rent** *B". SA12(f): std dev = 1/48* **day** *(= ½ hr).*

23 · Final checklist before writing an answer

- *Same time unit everywhere (convert min to hr first).*
- *Listed EVERY resource? Shared activities added to EVERY resource that does them? Pools multiplied?*
- *Pooled activity counted once in flow time? Parallel branches: longest path?*
- *Utilization uses throughput; implied utilization uses input.*
- *Inventory never below 0; found when a shrinking queue reaches 0; divided by the whole time window?*
- *Little's Law: l, R, T for the same part (queue vs whole system)?*
- *WSPT: ranked by $p \div w$ but added real p for C?*
- *Queue: ρ = λ/μ or λ/(cμ), checked < 1? Right CV (exponential 1, constant 0, else std dev ÷ mean)? "In system" ⇒ added service?*
- *Write: model → formula → numbers → answer with units, rounded to 2 decimals.*

24 · All formulas at one glance

Quantity	Formula	Unit
Flow time (one line)	sum of activity times	min
Flow time (parallel)	longest START→END path	min
Unit load of resource	sum of times of all its activities	min/unit
Capacity of resource	60 ÷ unit load (× pool size)	units/hr
Process capacity	smallest resource capacity	units/hr
Throughput	min(input, process capacity)	units/hr
Utilization	throughput ÷ resource capacity	% (≤ 100)
Implied utilization	input ÷ resource capacity	% (any)
Cycle time	1 ÷ throughput	hr/unit
Queue (discrete)	max(0, previous + input − capacity)	units
Slope (continuous)	input − capacity	units/hr
Average inventory	area ÷ time, or Σ snapshots ÷ n	units
Little's Law	$l = R \times T$	—
Completion time	$C_i = W_i + p_i$	min
SPT / WSPT order	smallest p / smallest $p \div w$ first	—
Mean, variance	$\Sigma\, x{\cdot}P$; $\Sigma\,(x - \text{mean})^2{\cdot}P$	x, x^2
CV	std dev ÷ mean	none
Exponential (rate r)	mean = std dev = 1/r, CV = 1	time
Poisson (mean m)	variance = m, std dev = $\sqrt{m}$	count
Utilization (queue)	$\rho = \lambda/\mu$; c servers: $\lambda/(c\mu)$	0–1
PK, 1 server	$\rho^2/(1-\rho) \times (C_A^2+C_S^2)/2$	people
PK, c servers	$\rho^{\sqrt{2(c+1)}}/(1-\rho) \times (C_A^2+C_S^2)/2$	people
Wait in queue	$T_q = l_q \div \lambda$	hr
In service	$T_s = 1/\mu$; $l_s = \lambda/\mu$	hr ; people
Total in system	$l = l_q + \lambda/\mu$; $T = T_q + 1/\mu$	people ; hr